

RESEARCH ARTICLE

From Synthetic Data to Real-World Efficiency: Automated Vehicle Damage Detection and Vehicle Parts Segmentation

CÁTIA LOUREIRO¹, SANDRA DIXE¹, CRISTIANA NETO², ANTÓNIO SILVA¹,
BEATRIZ MIRANDA¹, RICARDO RODRIGUES¹, (Member, IEEE), NUNO M. C. DA COSTA¹,
CARLOS FERREIRA³, (Member, IEEE), HUGO PEIXOTO², JOSÉ MACHADO²,
AND DUARTE FERNANDES¹

¹DTx-Digital Transformation CoLab, University of Minho, Azurém Campus, 4800-058 Guimarães, Portugal

²LASI, Algoritmi Center, University of Minho, 4710-057 Braga, Portugal

³Electronics and Connectivity (EC) Team, CEiiA, 4450-017 Matosinhos, Portugal

Corresponding author: Cátia Loureiro (catia.loureiro@dtx-colab.pt)

This work was supported by European Union under the NextGenerationEU, through a grant from Portuguese Republic's Recovery and Resilience Plan (PRR) Partnership Agreement, within the scope of the project Be.Neutral-Agenda da Mobilidade para a Neutralidade Carbónica das Cidades under Project 35-C644874240-00000016.

ABSTRACT The growing urbanization of modern societies intensifies the demand for sustainable mobility that aligns efficiency, safety, and environmental responsibility in smart cities. Shared vehicle systems address this need, but remain limited by subjective manual inspections that hinder operational efficiency. Automated exterior damage assessment via computer vision offers a scalable alternative, although its success critically depends on high-quality, domain-specific datasets, which remain scarce. To overcome this limitation, we introduce SYNDCAR (SYNthetic Damage CAR dataset), a generative AI-based dataset that simulates realistic vehicle damage in images with precise annotations for detection and vehicle parts segmentation. Extensive experiments were conducted that covered hyperparameter tuning, transfer learning, benchmarking of CNN- and Transformer-based models, and post-training optimization and quantization. The results highlight the challenges of damage detection due to appearance variability and dataset size, while pointing to practical solutions: YOLOv8-X achieved the highest accuracy, whereas YOLOv12-S provided the best trade-off. TensorRT deployment tests (FP16/INT8) further reduced the inference time by $\approx 71\%$ on the GPU and the energy consumption by 65%. Segmentation remained more energy-intensive than detection. In general, SYNDCAR demonstrates the potential of synthetic datasets to enable accurate, efficient, and scalable vehicle inspection systems.

INDEX TERMS Benchmarking, computer vision, object detection, quantization, segmentation, synthetic dataset.

I. INTRODUCTION

The accelerating pace of automation in manufacturing, coupled with global population growth, has led to a substantial increase in privately owned vehicles. This expansion of individual mobility directly contributes to increased traffic congestion, higher incidence of traffic accidents, and

escalating carbon emissions [1]. These factors not only place pressure on urban infrastructures, but also directly contribute to environmental degradation and climate change [2]. Hence, the rapid urbanization of contemporary societies has accelerated the pressing and global demand for sustainable mobility solutions that balance efficiency, safety, and environmental responsibility within the framework of emerging smart and green cities [3]. Shared vehicle fleets have emerged as a promising alternative, optimizing the use of transportation

The associate editor coordinating the review of this manuscript and approving it for publication was Anandakumar Haldorai¹.

assets while reducing the number of vehicles that circulate in dense urban areas [4]. In this context, shared mobility systems have the potential to reduce traffic congestion, decrease greenhouse gas emissions, and optimize the use of limited urban space by reducing the number of privately owned vehicles [5].

The effectiveness of shared fleets, however, relies heavily on timely and reliable management processes, particularly in the detection and maintenance of exterior vehicle damage. Conventional manual inspection methods are time-consuming and inherently prone to subjectivity and disputes between users and fleet operators. In the context of shared mobility systems, where vehicles are accessed by multiple users daily, such inefficiencies are amplified, often leading to operational delays, increased management overhead, and poor user experience, thereby reducing user trust in shared mobility services [6].

Advancements in Computer Vision (CV) and Deep Learning (DL) have provided new opportunities to automate visual inspection tasks with high precision and consistency [6]. Integrating such technologies into urban mobility infrastructures aligns with the principles of smart city development, where Artificial Intelligence (AI), automation, and data-driven decision-making are used to improve efficiency and sustainability. Automatic vehicle damage detection has the potential to significantly accelerate damage inspection workflows, provide more accurate estimations of repair scope, offer higher fleet availability, and support vehicle safety monitoring by identifying defects that may compromise vehicle performance.

As part of the Be.Neutral initiative, whose aim is to position Portugal as a global exporter of zero-carbon cyber-physical mobility solutions, this study addresses the problem of exterior damage detection in shared vehicle fleets. Specifically, the focus is on identifying visible anomalies such as broken glass, broken lights, cracks, and scratches, through the application of state-of-the-art computer vision techniques.

A central challenge in this study lies in data acquisition, which must balance the demands of realism, diversity, and ethical considerations. Creating real physical damage on vehicles for the sole purpose of data collection is an impractical and ethically questionable task, as it would involve deliberate infliction of harm on vehicles. The most viable alternative to overcome this limitation is to use generative AI to synthesize realistic damage directly onto vehicle images. This strategy ensures that a sufficiently varied and representative dataset can be produced without incurring the financial costs, operational risks, or ethical implications associated with physical damage creation. The resulting images were subsequently finely annotated, forming the basis for training and evaluating state-of-the-art DL models capable of automating and refining the detection of exterior vehicle damage.

This work extends a previous study [7], which evaluated vehicle damage detection and parts segmentation using publicly available datasets; that study addresses the identified

shortcomings of limited diversity and inconsistent annotations. To this end, the SYND CAR dataset was developed, which offers high-quality fine-grained annotations for both damage detection and parts segmentation.

In summary, this work proposes an automated framework to automatically detect and locate damage on vehicle surfaces, thereby reducing reliance on manual inspection and supporting the sustainable operation of fleet-based mobility platforms. Situating vehicle damage detection within the context of smart city and intelligent transportation ecosystems, this study contributes to the creation of greener, safer, and technologically advanced urban mobility infrastructures that balance efficiency, safety, and sustainability.

The remainder of this paper is structured as follows: Section II reviews related work on automated vehicle inspection. Section III elaborates on the processes of data acquisition, generation of synthetic damage, and procedures for image annotation. Section IV delineates the methodological framework employed during the experimental phases. Section V elucidates the findings from the experiments conducted and provides an analytical interpretation of the results. Section VI presents a critical discussion of the findings and outlines potential avenues for future research. Finally, Section VII synthesizes the conclusions drawn from this study.

II. RELATED WORKS

Automatic detection and localization of vehicle surface damage is an emerging field that leverages advancements in image processing techniques, DL methodologies, and sensor technologies. Recent studies indicate significant progress in the application of convolutional neural networks (CNNs) to assess vehicle damage, demonstrating the effectiveness of deep neural networks in this domain [8].

Dhieb et al. [9] utilized Inception-ResNetV2 and Mask R-CNN for insurance claim applications, achieving 86.8% precision in vehicle damage localization and suggested enhancing the model with diverse data and real-time optimization. Similarly, Jameel et al. [10] applied Mask R-CNN with background subtraction and HOG features, achieving 93.88% accuracy in vehicle segmentation, while recommending improved robustness, multi-sensor integration, and real-time applicability. Thonglek et al. [11] employed CapsNet for damage detection, achieving 91.53% localization accuracy and 98.47% classification accuracy, and proposed integrating driver applications and expanding repair databases.

More recently, Venkatesh et al. [12] compared different architectures — VGG16, YOLOv8, and SegFormer — for real-time damage assessment in insurance claims. Their study highlighted the complementary nature of classification models such as VGG16, which are effective in identifying whether a vehicle is damaged, and object detection models such as YOLO (You Only Look Once), which excel in localizing specific damaged regions. The authors achieved 92% accuracy in general, and emphasized the importance of extending datasets and building end-user

applications for practical adoption. Amodu et al. [13] further explored generative adversarial networks (GANs), shallow CNNs, and deep neural networks (DNNs), finding that CNNs achieved the best performance (91% accuracy) while recommending dataset expansion and hybrid model approaches.

Despite these advances, one of the most significant obstacles remains the lack of standardized, annotated, and high-quality training data. The existing datasets for the assessment of car damage have several limitations. Most are relatively small in scale, and many are not publicly accessible, limiting reproducibility and impeding fair comparisons between competing approaches. In addition, the available annotations are often coarse or incomplete, typically lacking detailed information such as precise damage localization, segmentation masks, or multiple damage categories. Image resolution and diversity are also frequently constrained, which reduces the effectiveness of datasets for training and evaluating contemporary DL models. Consequently, most existing datasets are primarily suited for basic damage classification tasks rather than allowing for more comprehensive and fine-grained analyses [14].

Among the existing resources, CarDD stands out as one of the most comprehensive and influential datasets developed for vehicle damage assessment. It comprises a relatively large collection of high-resolution images accompanied by detailed annotations that support multiple computer vision tasks, including damage classification, object detection, instance segmentation, and salient object detection [14].

Nevertheless, the dataset presented in this study extends this line of research by addressing several limitations. Introduce dual-level annotations that encompass both vehicle parts and damage types, enabling a more fine-grained understanding of the spatial and contextual relationships of vehicle damage. Moreover, the majority of images in the dataset are captured from a distant viewpoint, more accurately reflecting real-world monitoring scenarios such as parking-lots or roadside surveillance. Finally, it integrates highly realistic synthetic damage generation, allowing controlled evaluations of model robustness and generalization when exposed to artificial perturbations. Hence, these features establish the present dataset as a more comprehensive, versatile, and practically relevant benchmark for advancing research on automated vehicle damage assessment.

III. THE SYNDCAR DATASET

The SYNDCAR (SYNthetic Damage CAR) [15] was created as a synthetic dataset using generative AI to simulate realistic exterior damage directly in vehicle images. The dataset enables controlled experimentation and supports both damage detection and vehicle parts segmentation tasks. This section describes the dataset construction process in detail, covering data acquisition, synthetic damage generation, annotation methodology, and key statistical properties that define the dataset.



FIGURE 1. Example images from the SYNDCAR dataset.

A. DATASET CONSTRUCTION

Data acquisition was performed to establish a robust and diverse foundation for the generation of synthetic automotive damage data. Each image shows a single Citroën Ami, captured from various angles and conditions within urban and university environments in Guimarães, Portugal. Five unique devices, featuring a range of sensors and resolutions, were employed. Each device acquired images from four predefined viewpoints (front, rear, lateral, and diagonal). Representative examples of the captured images are shown in Fig. 1.

A total of 400 images were collected, of which 245 were retained after rigorous manual curation process. The excluded criteria included blurriness, underexposure, occlusion, and duplication. In addition, due to practical constraints in data acquisition and subsequent curation, neither the initial nor the final image sets are uniformly distributed across acquisition devices or viewpoints.

The final images were organized by device, with filenames that included a device identifier and timestamp to ensure traceability.

B. SYNTHETIC DAMAGE GENERATION

This subsection details the methodology used to generate synthetic damage to the captured images. It encompasses the selection and preparation of training images, design of prompts for the generative model, and subsequent procedure for damage generation.

1) TRAINING IMAGES

In this study, four representative types of automotive damage were selected for synthetic generation: broken glass, broken lights, cracks, and scratches. Close-up and visually diverse images that illustrate each of these types of damage were carefully selected from publicly available internet sources and proprietary photographs of various vehicles. On average, 20 representative images per class were used for fine-tuning, with a minimum of 10 images per class to ensure sufficient visual variability within each type of damage. All images were standardized by resizing and padding to a square aspect ratio of 768×768 pixels, following the input dimensions

recommended for Stable Diffusion (SD) version 2.1 [16]. Each image was also accompanied by a binary mask that accentuated the damage region, which served to guide the model’s attention during fine-tuning.

2) PROMPT DESIGN

The prompts describing each image were meticulously crafted, favoring concise and unambiguous language that emphasized the essential characteristics of each damage type. Various prompt formulations were explored during development; the most effective approach consisted of detailed single-sentence descriptions that consistently incorporated a unique instance token (e.g., “ohwx damage”) as an explicit identifier for the target concept. The “ohwx” token, commonly used in SD prompts, served to uniquely label each type of damage during training, thereby facilitating the precise identification and differentiation of styles or subjects. Furthermore, consistent use of relevant keywords on all the prompts further supported the learning process.

An example of an effective prompt used in this work is: “A close-up photo of a gray car with ohwx damage: the car shows multiple small scratches near the tire.”

3) TRAINING PARAMETERS

To fine-tune the SD model, training was conducted using the OneTrainer tool [17]. The learning rate scheduler was set to *Constant*, and a learning rate of 3×10^{-6} was applied equally to both the UNet and text encoder. The model was trained for 200 epochs with a batch size of 4. Masked training was enabled, meaning that loss computation focused more heavily on the masked (damaged) regions of each image. Specifically, the unmasked weight parameter was set to 0.6, so the loss on undamaged (unmasked) areas contributed 60%, while the loss on damaged (masked) regions contributed to full weight (100%). This weighting scheme encourages the model to prioritize learning on the target areas of synthetic damage while preserving visual consistency in the remainder of the image.

To mitigate catastrophic forgetting and preserve the general knowledge of vehicle appearance, 500 regularization images of various car types were incorporated during fine-tuning. The repeat factor was adjusted to ensure that the number of regularization images matched the number of training samples in each epoch. This approach prevented all 500 regularization images from being used in a single epoch and ensured balanced exposure across the training iterations.

4) GENERATING DAMAGES

Damage generation was systematically conducted using the Automatic1111 web UI [18] for SD. The image-to-image inpainting tool enabled the targeted creation of four distinct damage types, ensuring the integrity of the original image outside the specified damage region. For each image and damage type, four variants were generated per batch, with the most appropriate result selected for subsequent analysis.

TABLE 1. Main hyperparameters used for stable diffusion-based damage generation.

Parameter	Value
Steps	20–30
Sampler	DPM++ 2M
Scheduler Type	Karras
CFG Scale	7
Denoising Strength	0.6–0.8 (prominent damage) 0.4–0.6 (subtle damage)



FIGURE 2. Representative examples of synthetic damage types applied to the Citroën AMI in the SYNDCAR dataset.

Multiple configurations of the generation parameters were evaluated and the following settings were found to produce the most consistent and satisfactory results (Table 1).

Fig. 2 presents representative examples of each generated damage class: broken glass, broken lights, cracks, and scratches.

C. IMAGE ANNOTATION

All images in the dataset were manually annotated for two separate computer vision tasks: object detection and instance segmentation. Annotations for both tasks were created using CVAT (Computer Vision Annotation Tool) [19] to ensure consistency and accuracy throughout the dataset.

The inclusion of vehicle parts segmentation alongside damage detection was a deliberate methodological decision to improve the contextual understanding of each detected anomaly. Although damage detection alone identifies the presence and location of surface defects, segmentation enables the association of these damages with specific structural components (e.g. doors, windows, lights, or mirrors). This dual-task design supports finer-grained analyses, such

as estimating repair costs or prioritizing maintenance actions, and provides a richer foundation for multi-task learning models that can jointly explain the damage type and its affected area. Consequently, segmentation plays a crucial role in moving beyond binary detection to a more comprehensive and actionable vehicle inspection framework.

For the object detection task, bounding boxes were drawn around each instance of the target damage class. These annotations were exported in the *Ultralytics YOLO Detection 1.0* format.

For the segmentation task, polygons were manually traced to precisely delineate vehicle regions. A total of 28 segmentation classes were defined, comprising distinct structural components of the vehicle (e.g., side panels, windows, lights, etc.), along with their corresponding perspectives (e.g., left, right, front, and rear). This detailed annotation scheme allows for a comprehensive analysis of damage localization. The resulting segmentation polygons were exported in the *Ultralytics YOLO Segmentation 1.0* format.

Subsequently, all annotations were reviewed and analyzed using the FiftyOne tool [20] to identify and correct any mislabeling or inconsistencies.

D. DATASET STATISTICS

The final SYNDCAR dataset comprises 245 annotated images. From a distributional point of view, the object detection data is characterized by a moderate annotation density (mean of ~ 5.6 objects per image) and relatively balanced class frequencies across the four damage categories (see Fig. 3). All four damage classes exhibit extremely small relative areas, occupying, on average, less than 1% of the total image area, with broken glass being the most prominent ($0.65 \pm 0.70\%$). The small relative size of annotated objects emphasizes the importance of deploying models capable of robust small-object detection. The inclusion of a small subset of non-damaged samples further introduces class sparsity that can be leveraged to assess false positive sensitivity during inference.

In contrast, the segmentation dataset presents substantially higher annotation granularity, with 28 defined classes capturing fine-grained structural and positional components of the vehicle. This leads to pronounced inter-class variability in both spatial extent and instance frequency. While certain classes (e.g., rear panels) occupy relatively large pixel regions, others (e.g., mirrors) cover minimal areas. Moreover, the uneven distribution of class instances introduces a class imbalance, which poses additional challenges in learning robust representations. The summary statistics are detailed in Tables 2 and 3, respectively.

IV. EXPERIMENTAL METHODOLOGY

This section outlines the comprehensive workflow adopted in this study to develop and optimize DL models for vehicle damage detection and parts segmentation. The methodology encompasses hyperparameter optimization, transfer learning, fine-tuning, and model quantization.

TABLE 2. Summary statistics of the object detection dataset: image counts, annotations, and average relative area per class.

Metric	Value
Total images	245
Train / Val / Test split	172 / 37 / 36
Number of classes	4
Images with damages	231
Images without damages	14
Annotations	
Total bboxes	1295
Objects per image (mean $\pm$ std)	5.61 ± 1.48
Avg. % area per class	
Broken_Glass	$0.65 \pm 0.70\%$
Broken_Lights	$0.20 \pm 0.13\%$
Cracks	$0.21 \pm 0.18\%$
Scratches	$0.28 \pm 0.30\%$

TABLE 3. Summary statistics of the segmentation dataset: image counts, annotations, and per-class relative area coverage.

Metric	Value
Total images	245
Train / Val / Test split	171 / 49 / 25
Number of classes	28
Annotations	
Total polygons	3341
Polygons per image (mean $\pm$ std)	13.51 ± 3.57
Avg. % area per class	
Largest: Rear_Panel	$8.73 \pm 8.03\%$
Smallest: Right_Mirror	$0.14 \pm 0.06\%$

A. OVERVIEW OF EXPERIMENTAL GOALS

This study presents the SYNDCAR dataset and includes a comprehensive evaluation of multiple experimental setups. The primary objective is to identify the methods and algorithms that are best suited for these data, focusing on two key tasks essential for exterior damage assessment: (i) **damage detection** and (ii) **vehicle parts segmentation**.

Fig. 4 summarizes the study workflow, from SYNDCAR dataset development to the four experimental stages and the final real-world validation. The four experimental stages are:

- 1) **Hyperparameter Optimization:** Identify the training configuration that maximizes model performance, improve key metrics in the SYNDCAR dataset, and establish a set of baseline hyperparameters for subsequent experiments.
- 2) **Transfer Learning and Fine-Tuning Analysis:** Compare CNN- and transformer-based architectures to assess their adaptability to small, domain-specific datasets. This step is crucial to anticipate architectural challenges before proceeding with large-scale experiments.
- 3) **Comprehensive Benchmarking:** Quantify accuracy, computational cost, and energy consumption across different model families and sizes to select the most

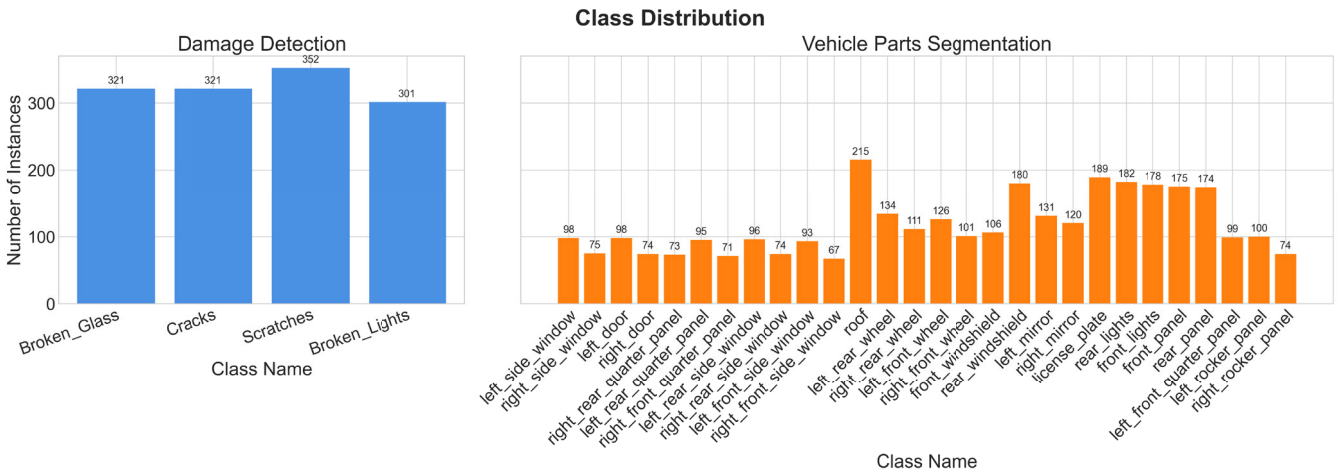


FIGURE 3. Class distribution statistics for the damage detection (left) and vehicle parts segmentation (right) task in the SYND CAR dataset.

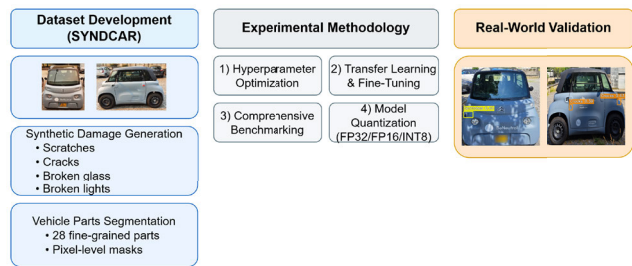


FIGURE 4. Graphical overview of the SYND CAR dataset development, experimental methodology (four stages), and real-world validation.

balanced solution for both detection and segmentation tasks.

- 4) **Model Quantization:** Evaluate the impact of Post-Training Quantization (FP32, FP16, INT8) on accuracy, inference time, and energy efficiency across multiple deployment frameworks (PyTorch, ONNX Runtime, TensorRT, OpenVINO), aiming to identify the most sustainable and deployment-ready solution.

B. HYPERPARAMETER OPTIMIZATION

Hyperparameter optimization was performed to improve both the training efficiency and detection performance by adjusting the parameters known to significantly influence the model’s learning dynamics. Although it is challenging to quantify the exact impact of each hyperparameter, previous studies and empirical evidence have highlighted the importance of careful tuning for better generalization [21]. The selection and adjustment of the hyperparameters were informed by theoretical insights and validated through experiments using the SYND CAR dataset.

Hyperparameter optimization was conducted exclusively on the YOLOv8 [22] model, which was selected as the representative baseline for this stage. This choice was motivated by two key factors. First, YOLOv8 provides a well-established and stable architecture that is extensively benchmarked

TABLE 4. Hyperparameters explored during the YOLOv8 optimization process.

Hyperparameter	Values Tested
Initial learning rate	$[1 \times 10^{-4}, 1 \times 10^{-3}, 1 \times 10^{-2}]$
Final learning rate	$[2 \times 10^{-3}, 5 \times 10^{-3}, 1 \times 10^{-2}]$
Momentum	$[0.6, 0.7, 0.8]$
Weight decay	$[1 \times 10^{-4}, 5 \times 10^{-4}, 1 \times 10^{-3}]$
Warmup epochs	$[3, 4, 5]$
Warmup momentum	$[0.6, 0.7, 0.8]$
Box loss gain (b _{ox})	$[0.05, 0.1, 0.5]$
Class loss gain (c _{ls})	$[0.1, 0.2, 0.5]$

across multiple computer vision tasks, making it a reliable reference for understanding the learning behavior of the SYND CAR dataset. Second, optimizing hyperparameters across all architectures would have imposed an impractical computational cost given the number of model variants and experimental combinations involved. Ultimately, the insights obtained from optimizing YOLOv8, including aspects such as learning rate scheduling, momentum, and loss weighting, were later adapted and implemented in other architectures during transfer learning and benchmarking. This approach ensured methodological consistency while maintaining the experimental efficiency.

The optimization process targeted eight key hyperparameters of the YOLOv8 model, as presented in Table 4.

Initially, the full hyperparameter search space was defined by specifying discrete candidate values for each parameter. A grid search strategy was used to evaluate all possible combinations, with each configuration trained for 10 epochs under consistent conditions. The search was executed using the Ray Tune optimization library [23], which allowed efficient parallelization of the experiments and substantially reduced total optimization time.

After each training run, the model’s performance on the validation set was assessed using the recall metric, and the results were logged into a.csv file for further analysis. Once all configurations were evaluated, the hyperparameter

combination that yielded the highest validation recall was selected. This configuration was subsequently used to retrain the YOLOv8 model with extended training epochs, and the final performance metrics were computed using the held-out test set.

C. TRANSFER LEARNING AND FINE-TUNING

Before the final benchmarking across model architectures, a targeted analysis of the transfer learning and fine-tuning requirements was conducted to evaluate the adaptability of different model types to the SYND CAR dataset. The comparison focused on two representative approaches: CNN-based and Transformer-based architectures. This focus was motivated by a well-known limitation of Transformer models: they generally require very large training datasets to achieve strong performance, whereas CNNs can often adapt more effectively with smaller datasets [24].

A common strategy to mitigate this limitation is to employ pretrained Transformer models trained on large general-purpose datasets and subsequently fine-tune them on smaller, domain-specific datasets. Therefore, this analysis examined whether a Transformer-based model (RT-DETR [31]) demands greater domain adaptation than a CNN-based alternative (YOLOv8) when applied to a small dataset. In the experiments, both models were initialized using COCO pretrained weights. Fine-tuning was applied in stages by progressively freezing network components, whereas transfer learning was tested by freezing most of the model and adapting only the final layers to the new domain.

D. MODEL BENCHMARKING

This subsection outlines the models adopted in this study for benchmarking. To ensure a systematic and representative evaluation, five object detection architectures (YOLOv8, YOLOv10, YOLOv11, YOLOv12, and RT-DETR) and two instance segmentation frameworks (YOLOv8 and YOLOv11) were included in the experimental setup. The selection criteria were primarily based on: (1) performance on the COCO dataset and (2) their current relevance in the field of real-time. The models chosen were as follows:

YOLOv8 [22] marks a significant step forward in the evolution of the You Only Look Once (YOLO) series. Unlike earlier anchor-based versions, it employs an anchor-free formulation inspired by FCOS and CenterNet detectors, thereby removing the dependence on manually defined anchor boxes. In addition, the prediction head was redesigned to align with contemporary single-stage detection frameworks, which simplified the post-processing pipeline, particularly non-maximum suppression (NMS). YOLOv8 also introduced a family of model sizes comprising the Nano (N), Small (S), Medium (M), Large (L), and Extra-Large (X) variants. When evaluated on the COCO benchmark, the largest variant (YOLOv8-X) achieved more than 53% Average Precision (AP) while maintaining real-time performance [25], confirming its effectiveness in combining detection accuracy and computational efficiency.

YOLOv10 [26], introduced in 2024, was designed to overcome several limitations related to the post-processing and architectural efficiency observed in previous YOLO versions. A key innovation of this model is the elimination of the NMS step, achieved through the adoption of a consistent dual-assignment strategy during training. This approach enables the network to learn both one-to-many and one-to-one label assignments, ensuring a consistent detection behavior between the training and inference phases. The architecture further incorporates an enhanced CSPNet backbone, a Path Aggregation Network (PANet) for improved multi-scale feature fusion, and dual detection heads. In terms of performance, YOLOv10-S demonstrates approximately $1.8\times$ higher inference speed compared to RT-DETR-R18, while achieving comparable AP on the COCO benchmark [25], highlighting its efficiency and precision balance.

YOLOv11 [27], also introduced in 2024, was developed to further enhance both the detection accuracy and computational efficiency. This version focuses on improving feature representation through an optimized backbone and neck architecture by integrating components such as the C3K2 block and SPPF module. These modifications strengthen the model's ability to capture multi-scale spatial information while maintaining low computational overhead. In terms of performance, YOLOv11 shows a higher mean Average Precision (mAP) with fewer parameters compared to previous YOLO variants. Specifically, YOLOv11-M surpasses YOLOv8-M on the COCO dataset [28], achieving superior detection accuracy while preserving the model compactness and inference efficiency.

YOLOv12 [29] represents a major architectural shift within the YOLO family, moving from purely convolutional designs toward an attention-oriented framework. Despite this transition, the model continues to deliver real-time inference performance, a defining feature of the YOLO lineage. At its core, YOLOv12 employs a lightweight self-attention mechanism that enables the efficient processing of extended receptive fields and enhances contextual feature understanding without incurring excessive computational cost. In addition, an updated feature fusion module, derived from the ELAN concept, was introduced to address the optimization difficulties commonly encountered in large-scale attention-based architectures. From a performance point of view, YOLOv12-X achieved an mAP of 55.2% on the COCO benchmark, an improvement of 0.6 percentage points over YOLOv11-X, while achieving a 4% reduction in inference time [30], underscoring its improved trade-off between accuracy and efficiency.

RT-DETR [31], a Real-Time Detection Transformer, is an end-to-end object detection framework that combines transformer-based modeling with real-time processing capabilities. Building on the foundation of DETR, a non-NMS-based architecture, RT-DETR introduces a convolutional backbone and a hybrid encoder specifically optimized to enhance inference speed while preserving high detection

accuracy. The model efficiently processes multi-scale visual representations by separating intra-scale interactions from cross-scale feature fusion, thereby enabling a more balanced trade-off between accuracy and computational cost. As an anchor-free detector, RT-DETR simplifies the detection pipeline and demonstrates an improved generalization across diverse datasets. In experimental evaluations, RT-DETR-L achieves an AP of 53% on the COCO benchmark, while maintaining a real-time throughput of 114 FPS on an NVIDIA T4 GPU (Graphics Processing Unit) [32], illustrating its strong balance between precision and efficiency.

In summary, the benchmark suite was constructed to capture the principal architectural trends currently defining real-time object detection and segmentation. YOLOv8 and YOLOv11 serve as robust CNN baselines, featuring mature anchor-free detection heads and scalable model families. Meanwhile, YOLOv10 and RT-DETR exemplify NMS-free paradigms, which streamline inference and enhance throughput by eliminating post-processing steps. YOLOv12 augments these baselines by incorporating lightweight attention mechanisms to improve contextual reasoning with minimal computational overhead. Together, these models cover a spectrum of accuracy–efficiency trade-offs, enabling a clear assessment of how architectural decisions impact deployment-relevant metrics.

E. MODEL COMPRESSION

Deep learning models are often computationally demanding, which limits their direct deployment on resource-constrained devices. Model optimization techniques address this challenge by reducing the size of the model and the computational cost while preserving accuracy. Among these, quantization stands out as a widely used and effective approach.

1) MODEL QUANTIZATION

Quantization in DL is a model compression technique that reduces the numerical precision of weights and activations [33]. By lowering the precision, memory usage is reduced, inference is speeded up, and power consumption is reduced. Two primary strategies are commonly adopted:

- 1) **Post-Training Quantization (PTQ):** In this approach, quantization is applied after the model is fully trained. Typically, the model parameters initially represented in FP32 (32-bit floating point) are converted to lower-precision formats such as FP16 (16-bit floating point) or INT8 (8-bit integer). It is fast to implement and well suited for adapting pretrained models to resource-constrained environments [34].
- 2) **Quantization-Aware Training (QAT):** In this method, quantization is integrated into the training process. QAT simulates the quantization effects on weights and activations, enabling the model to adjust during learning. As a result, models trained with QAT often achieve higher accuracy than those quantized with PTQ [34].

In this study, a PTQ approach was adopted to compress the models. PTQ enables runtime efficiency improvements with only modest reductions in accuracy, and is particularly relevant for deployment on hardware with limited resources. However, in practice, moving from development to deployment also requires selecting the most suitable inference framework for target hardware. Although general-purpose libraries are highly effective for training and experimentation, they are not always the most efficient option for real-time or energy-constrained scenarios.

To address this issue, four deployment-oriented frameworks were evaluated: PyTorch, ONNX, TensorRT, and OpenVINO. Although PTQ provides an effective means of reducing model complexity, the choice of deployment framework ultimately defines how well the models balance accuracy, inference speed, and energy consumption. Since sustainability is a key concern, this study places particular emphasis on performance and energy efficiency.

2) CONVERSION WORKFLOW

The workflow for creating and deploying DL models in this study follows a structured pipeline. After training the models, the weights were saved. To enable cross-platform deployment and inference optimization, trained models were exported to ONNX, a framework-agnostic format that facilitates interoperability with multiple inference engines. The ONNX models were then optimized using frameworks such as TensorRT and OpenVINO with different numerical precisions, including FP32, FP16, and INT8 (dynamic or static quantization), to accelerate inference, reduce memory, and adapt to heterogeneous hardware. During inference, outputs undergo post-processing, where redundant or overlapping predictions are removed through NMS, retaining only the most confident predictions.

For evaluation, both the model predictions and the ground-truth annotations are converted to the COCO dataset format. This standardization ensures uniform comparison between different models, frameworks, and precisions, allowing the consistent computation of metrics.

a: EXPORT AND QUANTIZATION DETAILS

PyTorch (FP32 baseline). Native PyTorch inference in FP32 was used as the reference configuration for all comparisons.

ONNX (FP32/FP16/INT8). Models were exported with the Ultralytics exporter using `dynamic = False`, `simplify = True`, and `opset = 12`. FP32 and FP16 variants were generated with `half=False` and `half = True`, respectively. Inputs were resized to 640×640 and normalized to $[0, 1]$ to match the evaluation pipeline. For ONNX INT8, dynamic PTQ was applied with ONNX Runtime using `quantize_dynamic(weight_type = QUInt8)`, which quantizes weights without requiring a calibration set.

TensorRT (FP32/FP16/INT8). TensorRT engines were generated from ONNX using NVIDIA `trtexec`. FP32 engines used the default builder configuration, FP16 enabled

half precision via `-fp16`, and INT8 used calibration-based PTQ (static INT8) via `-int8` with a calibration image directory specified by `-calib <calib_images_dir>`. Engines were serialized as `.engine` files for deployment and benchmarking.

OpenVINO (FP32/FP16). Models were exported to the OpenVINO Intermediate Representation (IR) using the Ultraalytics OpenVINO backend (`format="openvino"`). FP32 and FP16 artifacts were generated with `half=False` and `half=True`, respectively, producing IR files (`.xml+.bin`) for benchmarking.

b: POST-PROCESSING AND EVALUATION PROTOCOL

For detectors producing overlapping predictions, NMS was applied to retain only the highest-confidence outputs. Predictions and ground-truth labels were converted to COCO format, ensuring consistent evaluation across backends and precision modes.

V. EXPERIMENTAL RESULTS AND ANALYSIS

This section presents an experimental evaluation of the proposed models and optimization strategies. The analysis covers the accuracy, inference time, and energy consumption across different architectures, frameworks, and quantization levels.

A. EXPERIMENTAL SETUP

All experiments were conducted using the SYNDCAR dataset, which contains annotated images of vehicle parts and exterior vehicle damage.

1) IMPLEMENTATION DETAILS

All models were developed using Python 3.10.11 in combination with the PyTorch 2.0.1 deep learning framework. Parallel processing was supported by CUDA 11.8 and the CUDNN library for neural network optimization. The experiments were executed on a Windows 10 system, equipped with an AMD Ryzen 9 3900X 12-core processor and NVIDIA GeForce RTX 2080Ti GPU.

2) TRAINING PROTOCOL AND DATA PREPARATION

All experiments followed a unified training protocol to ensure comparability across models and tasks. The SYNDCAR dataset was split into training, validation, and test sets using a fixed ratio (Section III-D). Input images were resized to 640×640 pixels, and a consistent preprocessing pipeline was applied across object detection and instance segmentation experiments.

Data augmentation was applied on-the-fly during training. Color-based augmentations included random adjustments to hue ($\pm 1.5\%$), saturation (up to 70%), and brightness/value (up to 40%). Geometric augmentations consisted of random horizontal flipping with a probability of 50%, random scaling with a factor of up to 0.5, and random translations of up to 10% of the image dimensions. Additionally, mosaic augmentation was enabled for all training samples. Models

were trained for up to 400 epochs with a batch size of 1, using early stopping with a patience of 50 epochs. Optimization-related hyperparameters, including learning rate, weight decay, momentum, and loss weighting, were determined through a dedicated hyperparameter optimization stage (Section V-B) and consistently applied during transfer learning and benchmarking.

3) EVALUATION INDICATORS

In this study, the detection and segmentation performance were evaluated using standard metrics such as precision (1), recall (2), Average Precision (3), and mean Average Precision (mAP) (4).

Precision measures the proportion of correctly identified positive instances among all predicted positives, whereas recall quantifies the proportion of correctly identified positives among all actual positives. Together, they provide complementary information on the trade-off between false positives and false negatives.

$$\text{Precision} = \frac{TP}{TP + FP} \quad (1)$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad (2)$$

For a more comprehensive evaluation, the Average Precision (AP) was computed as the area under the precision-recall curve for each class i , given by:

$$AP_i = \sum_{k=1}^K P(k) \cdot \Delta r(k) \quad (3)$$

where $P(k)$ denotes the precision k , and $\Delta r(k)$ represents the change in recall.

Finally, mAP is obtained by averaging the AP values across all N classes:

$$\text{mAP} = \frac{1}{N} \sum_{i=1}^N AP_i \quad (4)$$

In addition to aggregate mAP, we report COCO size-specific Average Precision (AP_M , AP_L), which measures performance on medium and large objects based on pixel area thresholds.

The energy consumption of the models was assessed by measuring both the energy of the baseline system and the energy used during inference [35]. The baseline energy (BE) was recorded over a period of 3 min of idle operation, without any computational load, while the total energy (TE) was measured during 3 min of continuous model inference. From these measurements, the energy per inference request was obtained. E_{excR} was calculated as the difference between TE and BE divided by the number of requests (NR) processed within the inference period. All results are reported in milliwatt-hours (mWh). This approach enables the quantification of the actual energy cost of model execution while properly accounting for the idle consumption

TABLE 5. Best hyperparameter configuration identified through tuning. Lists the optimal values that yielded the highest recall on the validation set.

Hyper-parameter	Optimal Value
Initial learning rate	1×10^{-3}
Final learning rate	2×10^{-3}
Momentum	0.6
Weight decay	1×10^{-4}
Warmup epochs	5
Warmup momentum	0.6
Box loss gain (box)	0.1
Class loss gain (cls)	0.5

of the system. The energy per execution request is calculated as follows:

$$E_{\text{excR}} = \frac{TE - BE}{NR} \quad (5)$$

Furthermore, the average inference time was quantitatively assessed and the utilization of both the CPU (Central Processing Unit) and GPU was systematically observed and recorded over the entire evaluation period. The computational throughput, expressed as Giga Floating Point Operations per Second (GFLOPs), was also measured to provide a more complete picture of the model’s efficiency.

B. EFFECT OF HYPERPARAMETER OPTIMIZATION

Hyperparameter optimization was conducted to refine the training configuration and assess its impact on the performance of the YOLOv8-S model trained on exterior damage images from the SYND CAR dataset. The optimization process, conducted using Ray Tune with a grid search strategy, identified the best-performing set of hyperparameters, as summarized in Table 5. Subsequently, these parameters were used to retrain the model using an extended training schedule.

Table 6 compares the performance of the baseline configuration (default) with that of the tuned model across all classes. At the aggregate level, hyperparameter tuning improved the precision from 0.738 to 0.768 and recall from 0.708 to 0.720, reflecting an increase in the model’s ability to correctly identify true positives and reduce false positives. However, both mAP^{50} and $\text{mAP}^{50:95}$ registered small declines, suggesting that improvements in classification confidence came at the cost of a slightly reduced localization accuracy.

At the class level, the results are more heterogeneous. Although “Broken Lights” benefited considerably from tuning, reaching the highest precision (0.871) and stable recall (0.902), classes such as “Cracks” and “Scratches” experienced mixed results. “Cracks” improved in recall (0.675 vs. 0.550) but suffered a decline in precision (0.553 vs. 0.586), leading to only a modest gain in mAP^{50} . Similarly, “Scratches” achieved a higher precision (0.873) but lower recall (0.724), reducing its overall localization accuracy.

Key Findings: Hyperparameter optimization led to global improvements in model precision and recall but did not

TABLE 6. Performance comparison between default and tuned YOLOv8-S models.

Classes	Precision	Recall	mAP^{50}	$\text{mAP}^{50:95}$
Baseline (Default)				
All	0.738	0.708	0.746	0.462
Broken Glass	0.766	0.590	0.675	0.329
Cracks	0.586	0.550	0.531	0.264
Scratches	0.849	0.788	0.837	0.428
Broken Lights	0.752	0.902	0.941	0.828
Tuned Model				
All	0.768	0.720	0.742	0.438
Broken Glass	0.775	0.580	0.607	0.273
Cracks	0.553	0.675	0.584	0.285
Scratches	0.873	0.724	0.828	0.407
Broken Lights	0.871	0.902	0.948	0.786

uniformly benefit all damage classes. In particular, the decrease in mAP values indicates a trade-off; whereas the tuned model was better at distinguishing positive cases, its bounding-box localization became slightly less accurate.

C. PERFORMANCE AFTER FINE-TUNING

In this study, a Transformer-based model (RT-DETR-L) was compared to a CNN-based architecture (YOLOv8-S). Previous research has shown that Transformer models are more prone to overfitting on smaller datasets than CNNs; however techniques such as transfer learning and fine-tuning can mitigate this challenge [36]. The investigation revealed distinct differences between the two architectural families, as delineated in Table 7.

For YOLOv8-S, moderate fine-tuning yielded only slight improvements compared with a fully frozen backbone, with marginal gains in $\text{mAP}^{50:95}$ (up to 0.448). In contrast, excessive freezing causes a substantial performance degradation. However, RT-DETR consistently outperformed YOLOv8-S across all the metrics. Its precision remained high and recall improved with deeper fine-tuning, reaching a peak when 20 layers were frozen (recall = 0.788, $\text{mAP}^{50:95}$ = 0.519). This suggests that Transformer-based detectors benefit more from controlled domain adaptation, with sustained performance gains even when additional layers are fine-tuned.

The inference times highlight the trade-off between the two architectures. YOLOv8-S was consistently faster on both the CPU and GPU, achieving ≈ 200 ms per inference on the GPU. RT-DETR, while exhibiting a comparable GPU speed, required nearly twice the CPU inference time (≈ 600 ms). This suggests that YOLOv8-S may be preferable for speed sensitive applications, whereas RT-DETR delivers superior detection accuracy when computational resources are available.

Key Findings: The experiments demonstrate that YOLOv8-S requires little to no fine-tuning to achieve stable performance, whereas RT-DETR benefits substantially from deeper fine-tuning, achieving higher recall and overall detection accuracy in small-dataset conditions.

TABLE 7. Performance of CNN- (YOLOv8-S) and transformer-based (RT-DETR) detectors after fine-tuning on SYNDCar.

Model	Frozen Layers	Precision	Recall	mAP ⁵⁰	mAP ^{50:95}
CNN-based					
YOLOv8-S	0	0.740	0.702	0.723	0.430
	5	0.722	0.712	0.730	0.448
	10	0.784	0.667	0.724	0.430
	15	0.774	0.687	0.724	0.422
	20	0.729	0.693	0.710	0.426
	22	0.353	0.388	0.343	0.204
Transformer-based					
RT-DETR	0	0.899	0.717	0.791	0.494
	5	0.864	0.742	0.798	0.498
	10	0.844	0.743	0.775	0.512
	15	0.863	0.747	0.787	0.519
	20	0.883	0.788	0.802	0.529
	28	0.838	0.782	0.800	0.515

D. MODEL BENCHMARKING

This section presents a comprehensive benchmark of the evaluated models considering three main aspects: accuracy, inference time, and energy consumption. These dimensions are crucial in assessing the trade-offs between performance and computational efficiency, which are fundamental when deploying models in real-world scenarios. Furthermore, it incorporates the key findings of Sections V-B and V-C, which detail the optimal hyperparameter configurations and the insights gained regarding layer freezing for both CNN-based and Transformer-based models.

1) ACCURACY

This subsection presents the evaluation results for damage detection (Table 8) and vehicle parts segmentation (Table 9).

The mAP⁵⁰ results indicate a substantial accuracy reduction for all exported pipelines relative to the PyTorch baseline. For damage detection, YOLOv8-X achieved the best performance among all the evaluated models, with an mAP⁵⁰ of 0.848 and an mAP^{50:95} of 0.533. In contrast, YOLOv10-N obtained the lowest results, with an mAP⁵⁰ of 0.624 and an mAP^{50:95} of 0.359.

In terms of the individual metrics, YOLOv12-L achieved the highest precision of 0.900, whereas YOLOv12-X achieved the highest recall of 0.797. Overall, the YOLOv10 family presented the weakest performance, but had a clear advantage in parameter efficiency. In contrast, YOLOv8, YOLOv11, and YOLOv12 showed comparable parameter counts, with YOLOv8 being slightly larger but delivering consistently high accuracy. RT-DETR exhibited a parameter count similar to that of YOLO-L variants, while offering competitive detection results. Regarding the computational complexity, the GFLOPs values scaled predictably with the model size ($N \rightarrow X$) and remained consistent in the YOLO versions.

For the segmentation of vehicle parts, only YOLOv8 and YOLOv11 were evaluated, and the results were overall very similar between the two models. YOLOv8-S obtained the

highest precision (0.945), while YOLOv11 achieved the best results in recall (0.951, L), mAP⁵⁰ (0.964, M) and mAP^{50:95} (0.829, M). In terms of model size, YOLOv11 requires slightly fewer parameters and has a lower computational cost (GFLOPs), which makes it more efficient while maintaining a competitive accuracy.

To evaluate robustness across different object scales, we report COCO scale-specific AP (AP_M and AP_L) in addition to the overall mAP, using the standard COCO area-based definition: medium if $32^2 \leq \text{area} < 96^2$ (in px^2) and large if $\text{area} \geq 96^2$ (in px^2). The ground-truth size distribution is dominated by large objects: for damage detection, the test split includes 35 medium and 153 large instances (total 188), while for vehicle parts segmentation it includes 15 medium and 314 large instances (total 329).

For damage detection, performance differences are more pronounced at larger scales. YOLOv8-X achieved the highest large-object performance ($AP_L = 0.512$) and the best medium-scale score ($AP_M = 0.310$), consistent with its top overall mAP⁵⁰ (0.848) and mAP^{50:95} (0.533). RT-DETR-L had the highest overall AP_L (0.540) and maintained a competitive mAP^{50:95} (0.510), indicating strong localization on large damage areas despite a slightly lower mAP^{50:95} than YOLOv8-X. Across model families, YOLOv10 variants consistently obtained lower AP_M and AP_L , matching the trend observed in global mAP while offering improved parameter efficiency.

For vehicle parts segmentation, YOLOv11-L achieved the best large-object score ($AP_L = 0.778$), closely followed by YOLOv8-L ($AP_L = 0.768$). Medium-scale performance remains substantially lower than AP_L across all variants (AP_M ranging from 0.189 to 0.325), indicating that the remaining errors are concentrated in medium-sized components rather than in large regions.

Key Findings: The model's benchmarking confirms that larger YOLO models and RT-DETR achieve higher accuracy but at the cost of greater computational complexity, whereas smaller models such as YOLOv10 offer reduced accuracy but clear advantages in efficiency. For segmentation, YOLOv8 and YOLOv11 performed similarly, with YOLOv11 showing a slight edge in efficiency.

2) INFERENCE TIME

This subsection presents the inference time in milliseconds for the damage detection and segmentation of vehicle parts (see Fig. 5).

The results reveal a clear performance gap between GPU-accelerated inference and CPU-only execution. On CUDA, lightweight models such as YOLOv8-N and YOLOv10-N achieved mean inference times below 12 ms and 10 ms, respectively, confirming their suitability for real-time applications. Larger variants, such as YOLOv12-X in detection (≈ 50 ms) and YOLOv8-X in segmentation (≈ 61 ms), exhibited an increased inference time, while RT-DETR-L maintained stable values of approximately 40 ms per inference.

TABLE 8. Benchmarking results for damage detection across different models. Lists accuracy metrics, parameter counts, and GFLOPs for each model version (Nano to Extra-Large).

Model	Version	Precision	Recall	mAP ⁵⁰	mAP ^{50:95}	AP _M	AP _L	#Params (M)	GFLOPS
YOLOv8	N	0.697	0.649	0.678	0.384	0.197	0.409	3.01	8.1
	S	0.789	0.673	0.737	0.426	0.237	0.423	11.14	24.4
	M	0.839	0.685	0.793	0.490	0.298	0.502	25.86	78.7
	L	0.794	0.760	0.825	0.480	0.302	0.498	43.63	164.8
	X	0.862	0.769	0.848	0.533	0.310	0.512	68.16	257.4
YOLOv10	N	0.770	0.569	0.624	0.359	0.228	0.395	2.71	8.2
	S	0.858	0.661	0.754	0.448	0.266	0.473	8.01	24.5
	M	0.893	0.710	0.781	0.459	0.277	0.472	16.49	63.4
	L	0.848	0.685	0.768	0.443	0.239	0.466	25.77	126.3
	X	0.827	0.693	0.767	0.430	0.241	0.456	31.66	169.8
YOLOv11	N	0.764	0.694	0.742	0.418	0.231	0.419	2.59	6.3
	S	0.859	0.691	0.779	0.459	0.243	0.472	9.43	21.3
	M	0.782	0.700	0.777	0.452	0.292	0.466	20.06	67.7
	L	0.779	0.748	0.772	0.482	0.300	0.493	25.31	86.6
	X	0.792	0.697	0.755	0.440	0.257	0.455	56.88	194.4
YOLOv12	N	0.806	0.627	0.740	0.421	0.171	0.447	2.57	8.6
	S	0.831	0.730	0.811	0.491	0.292	0.495	9.25	21.2
	M	0.815	0.709	0.779	0.490	0.286	0.487	20.14	67.1
	L	0.900	0.728	0.816	0.501	0.302	0.477	26.39	88.6
	X	0.814	0.797	0.811	0.485	0.268	0.493	59.12	198.5
RT-DETR	L (freeze 15)	0.889	0.742	0.830	0.510	0.292	0.540	32.81	103.4

Note: Under COCO’s absolute area thresholds, the evaluated split contains no “small” instances (area < 32² px); therefore AP_S is undefined and reported as N/A.

TABLE 9. Benchmarking results for vehicle parts segmentation across different models. Lists accuracy metrics, parameter counts, and GFLOPs for two model version (Nano to Extra-Large).

Model	Version	Precision	Recall	mAP ⁵⁰	mAP ^{50:95}	AP _M	AP _L	#Params (M)	GFLOPS
YOLOv8	N	0.930	0.945	0.953	0.799	0.281	0.755	3.27	12.0
	S	0.945	0.949	0.958	0.807	0.287	0.771	11.80	42.5
	M	0.925	0.950	0.952	0.816	0.317	0.774	27.26	110.0
	L	0.933	0.939	0.950	0.816	0.325	0.768	45.96	220.2
	X	0.933	0.933	0.944	0.792	0.273	0.753	71.78	343.8
YOLOv11	N	0.943	0.939	0.956	0.800	0.189	0.761	2.85	10.2
	S	0.919	0.930	0.950	0.801	0.241	0.757	10.09	35.4
	M	0.929	0.950	0.964	0.829	0.289	0.771	22.38	123.1
	L	0.935	0.951	0.952	0.821	0.238	0.778	27.64	142.0
	X	0.906	0.941	0.955	0.814	0.314	0.770	62.08	318.7

Note: Under COCO’s absolute area thresholds, the evaluated split contains no “small” instances (area < 32² px); therefore AP_S is undefined and reported as N/A.

On the CPU, inference time increases significantly. For example, YOLOv8-N required 55.7 ms for detection and 75.8 ms for segmentation, while YOLOv12-X and YOLOv11-X peaked at 717.9 and 696.6 ms, respectively. This consistent ×5 to ×15 highlights the computational load when the models were executed without GPU acceleration.

RT-DETR-L also remained above 370 ms across the CPU runs.

Across both tasks, model size directly impacted speed: Nano (N) and Small (S) models consistently delivered the lowest inference times, whereas Extra-Large (X) variants incurred the highest costs. Interestingly, YOLOv10 models

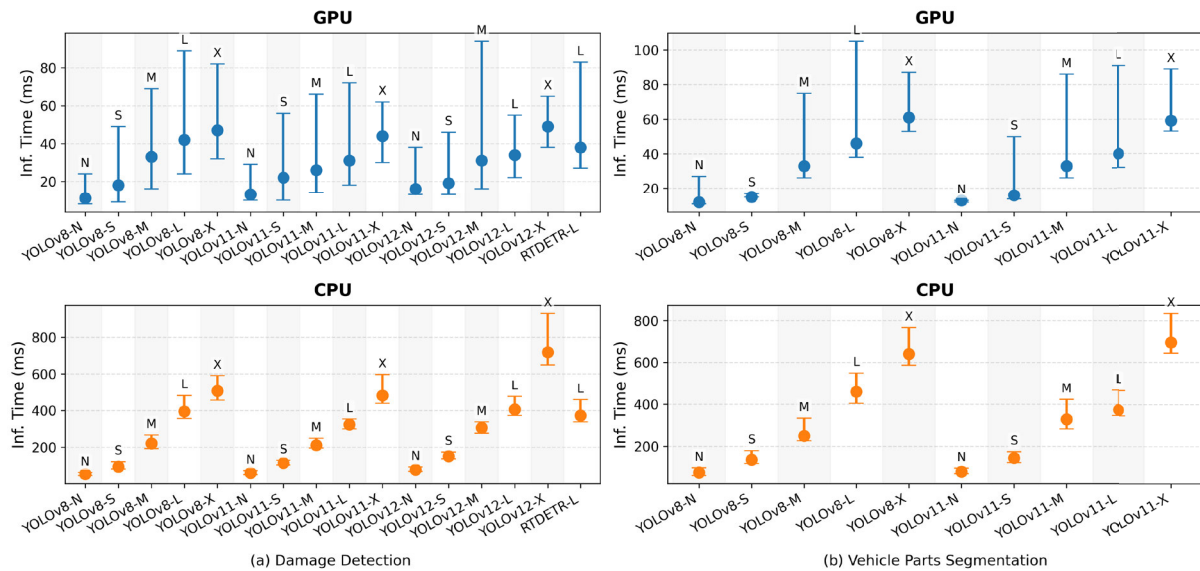


FIGURE 5. Inference time (ms) of different models on GPU and CPU for (a) damage detection and (b) vehicle parts segmentation, measured after a warm-up phase.

showed more efficient scaling than YOLOv8 in detection, while YOLOv11 achieved balanced performance in segmentation, with YOLOv11-N averaging just 13.1 ms on CUDA.

Key Findings: Inference time scales with model size, with lightweight models achieving low inference times and larger models incurring higher costs. GPU acceleration dramatically reduces speed, highlighting its importance for real-time deployment, whereas CPU execution remains feasible for smaller models or constrained environments.

3) ENERGY CONSUMPTION

The efficiency of the tested models was evaluated by measuring the energy per inference (ExecR, mWh) required to process benchmark datasets. Fig. 6(a) presents the results for damage detection and Fig. 6(b) for vehicle parts segmentation.

In the damage detection task, lightweight models such as YOLOv8-N and YOLOv10-N consumed less than 0.4 mWh, while larger models such as YOLOv8-X and YOLOv12-X exhibited significantly higher energy requirements, reaching 2.82 mWh and 3.03 mWh, respectively. Interestingly, YOLOv11-S achieved competitive balance, requiring only 0.50 mWh.

For the more computationally intensive vehicle parts segmentation task, the overall energy consumption was markedly higher in all the models. The smallest variant (YOLOv8-N) still consumed 1.42 mWh, which is almost four times higher than that in the detection task. Larger architectures, such as YOLOv8-X and YOLOv11-X, reached the highest values, with 4.24 mWh and 4.40 mWh, respectively. These results indicate that segmentation tasks are inherently more energy-intensive than object detection, even when models of comparable size are used. Overall, the results highlight a

clear scaling trend: larger models consistently yield higher energy consumption, which is exacerbated in segmentation workloads.

Key Findings: Energy consumption scales predictably with both the model scale and task complexity. Lightweight models achieve low per-inference energy, making them suitable for energy-constrained deployments, whereas larger models incur substantially higher energy costs. Vehicle parts segmentation is inherently more energy-intensive than damage detection, even for models of similar scales.

4) ACCURACY VERSUS INFERENCE TIME

The YOLO family of models has consistently pursued a balance between detection accuracy and computational speed, enabling real-time performance while gradually improving precision in successive versions [25]. As the architecture evolved, this trade-off became more refined, with different model scales being offered to accommodate diverse applications and hardware constraints. This subsection examines the relationship between accuracy (mAP⁵⁰) and inference speed (ms) for the two tasks under study.

For damage detection, the results in Fig. 7 show that YOLOv8-X achieves the highest mAP, but at the cost of increased computational time, exceeding 45 ms on the GPU and 500 ms on the CPU. In contrast, the smaller YOLOv8-N and YOLOv11-N variants delivered much faster inference (below 15 ms on the GPU and 100 ms on the CPU), although with reduced accuracy. Interestingly, RT-DETR achieves accuracy levels comparable to those of YOLOv8-L and YOLOv12-X while requiring less inference time.

In general, YOLOv12-S achieves the most favorable balance between accuracy and efficiency, surpassing 80% mAP with inference times under 20 ms on the GPU

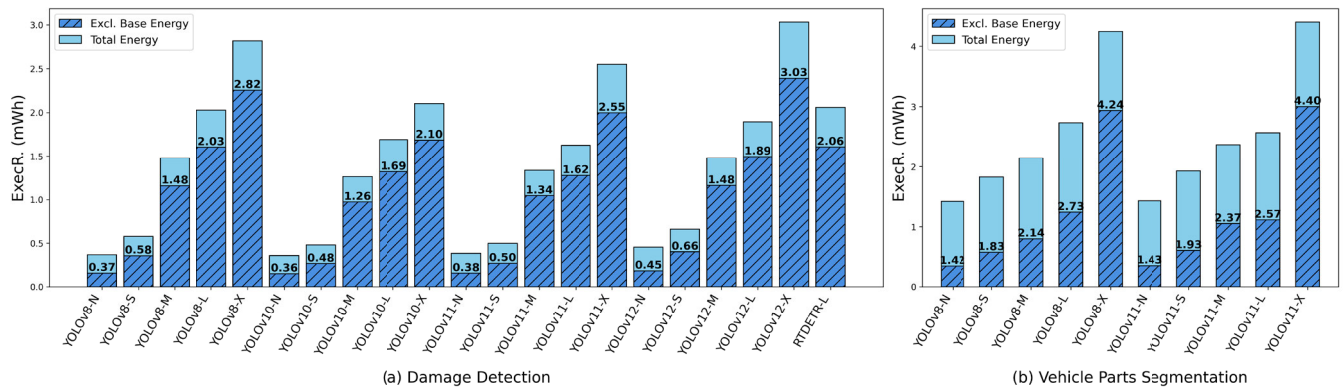


FIGURE 6. Comparative analysis of total energy consumption per request (ExecR, mWh) and baseline-excluded energy consumption per request (mWh) across different models on (a) the damage detection task and (b) the vehicle parts segmentation task.

and 200 ms on the CPU. These findings emphasize the continuous evolution of the YOLO family, where successive versions increasingly optimize the trade-off between speed and accuracy.

A similar pattern was observed in the results for the segmentation of vehicle parts. In this task, YOLOv11-M achieved the best trade-off, combining an inference time below 40 ms on a GPU and 400 ms on a CPU with an accuracy exceeding 96.25% mAP. However, unlike detection models, segmentation results reveal a general decline in accuracy as the model variants increase in size. In particular, YOLOv8-X recorded the lowest performance, with mAP values falling below 94.5% while incurring substantial computational costs (over 60 ms on the GPU and 600 ms on the CPU). In general, the YOLOv8 family performs poorly compared to YOLOv11, with the exception of the small variant, where YOLOv8-S achieves a slightly higher accuracy at nearly identical inference times.

Key Findings: Smaller models deliver fast inference but lower accuracy, whereas larger models achieve a higher mAP at the cost of increased inference time, particularly on the CPU. YOLOv12-S (detection) and YOLOv11-M (segmentation) provided the most favorable balance, combining high accuracy with moderate inference times.

E. COMPARISON OF FRAMEWORKS AND PRECISIONS

This subsection reports the experimental results for the models selected in Section V-D, which obtained the best trade-off. The damage detection task was addressed with YOLOv12-S, whereas vehicle parts segmentation was performed with YOLOv11-M. Both models were exported to three deployment frameworks and tested under different numerical precisions, allowing a direct comparison of their performance across platforms.

1) ACCURACY AND MODEL SIZE

The results presented in Table 10 provide a comprehensive overview of both the model size and the performance of mAP⁵⁰ in detection and segmentation tasks, considering four

deployment frameworks (PyTorch, ONNX, TensorRT, and OpenVINO) and three precision modes (FP32, FP16, and INT8).

The mAP results revealed a noticeable drop in accuracy for all exported models compared to the baseline PyTorch implementation. Specifically, detection performance decreased from 0.830 (PyTorch) to 0.462–0.518 across deployment frameworks, whereas segmentation performance decreased from 0.904 (PyTorch) to 0.706–0.708. Despite this reduction, some frameworks consistently outperformed others. TensorRT achieved the highest *deployed* mAP (0.518) for both FP32 and FP16 modes. In the segmentation task, mAP remained relatively stable across frameworks and precision modes, with ONNX (FP32) reaching the highest value (0.708) by a small margin.

Although TensorRT delivered the best mAP for detection, it did not achieve the smallest model size. The smallest model was obtained with ONNX INT8, which reduced its size to just 9.34 MB. In general, except for the INT8 quantization, most exported models were larger than the baseline model. A similar trend was observed for the segmentation models: ONNX FP32/FP16 reached 85.56 MB, and TensorRT FP32 reached 101.19 MB, compared to 43.15 MB for the PyTorch model.

This highlights a direct accuracy–size trade-off: ONNX INT8 minimizes footprint (9.34 MB) at the cost of lower detection mAP⁵⁰ (0.456), whereas TensorRT FP16 maximizes deployed mAP⁵⁰ (0.518) with a larger serialized model.

a: mAP DROP ANALYSIS

Input preprocessing was strictly identical across all backends, including PyTorch, using the same input resolution (640 × 640) and normalization. Consequently, the observed discrepancies cannot be attributed to input handling. Post-processing, however, differs between pipelines: ONNX Runtime, TensorRT, and OpenVINO outputs were processed using the same external class-aware NMS implementation, whereas the PyTorch baseline evaluated via Ultralytics relies on integrated post-processing.

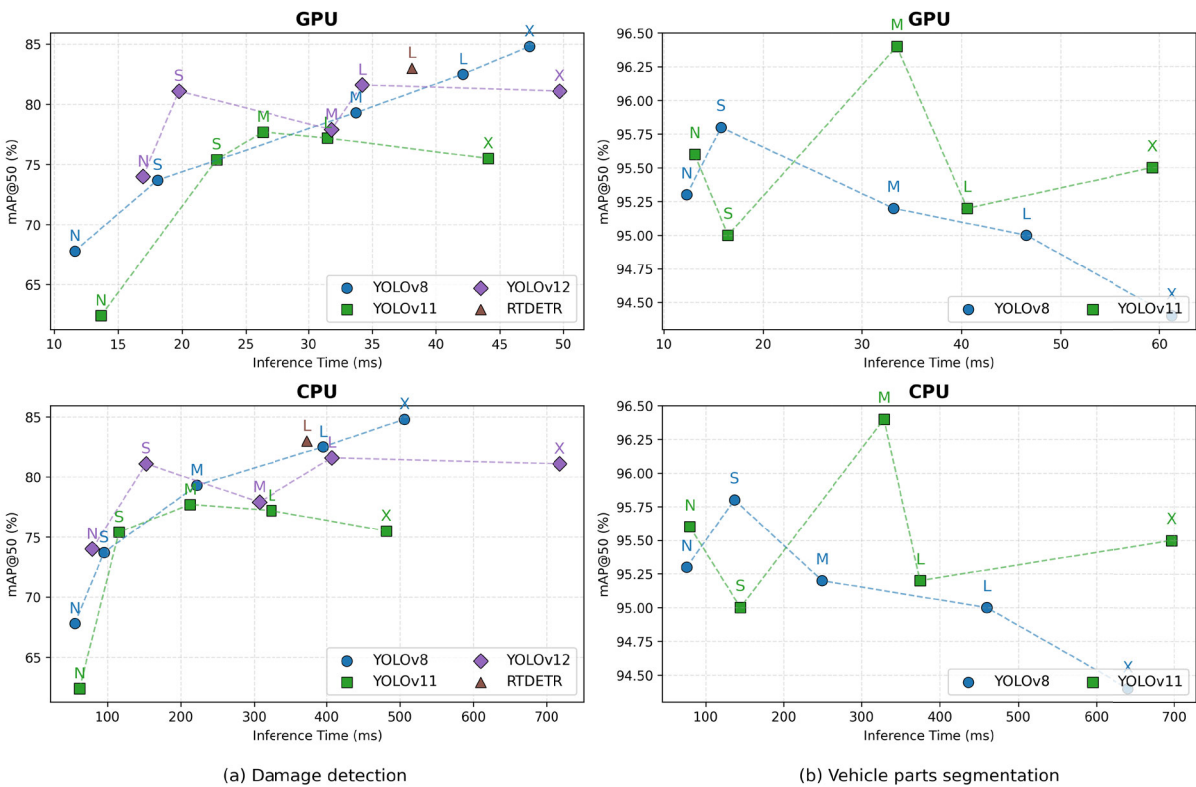


FIGURE 7. Comparative trade-off between inference time (ms) and detection accuracy (mAP⁵⁰, %) across different models on (a) the damage detection task and (b) the vehicle parts segmentation task, evaluated on both GPU and CPU.

TABLE 10. Performance comparison for Detection with YOLOv12-S (i) and segmentation with YOLOv11-M (ii) across different export frameworks (PyTorch, ONNX, TensorRT, and OpenVINO) and different precision models (FP32, FP16, INT8), considering the mAP⁵⁰ metric and model size in MB. TensorRT uses static INT8 quantization, while ONNX applies dynamic INT8 quantization.

		PyTorch		ONNX		TensorRT			OpenVINO	
		FP32	FP32	FP16	INT8	FP32	FP16	INT8	FP32	FP16
mAP ⁵⁰	(i)	0.830	0.462	0.462	0.456	0.518	0.518	0.467	0.462	0.462
	(ii)	0.904	0.708	0.707	0.706	0.707	0.707	—	0.707	0.707
Size (MB)	(i)	18.41	35.51	35.51	9.34	46.43	24.86	15.99	35.79	18.28
	(ii)	43.15	85.56	85.56	22.60	101.19	50.37	—	85.79	43.26

The impact of this NMS mismatch was quantified by applying the external NMS (used by deployment backends) to raw PyTorch outputs under the same thresholds (confidence = 0.25, IoU = 0.5). Replacing Ultralytics internal NMS with the external implementation reduced mAP⁵⁰ from 0.830 to 0.770 for detection (−0.060) and from 0.904 to 0.874 for segmentation (−0.030), indicating that post-processing differences account for a non-negligible portion of the gap observed in Table 10.

After controlling preprocessing and standardizing NMS across deployment backends, remaining differences are more plausibly linked to backend-dependent output decoding and runtime execution. In particular, confidence score semantics may vary across exported graphs and inference

implementations (e.g., logits vs. probabilities and objectness handling), such that identical numeric thresholds can lead to different filtering and ranking, directly affecting NMS outcomes. In addition, backend-specific graph optimizations and kernel implementations may introduce small numerical perturbations in scores and box/mask geometry that can be amplified by suppression.

To evaluate the role of ONNX operator-set selection, models were exported using opset versions 11, 12, 13, and 17 and evaluated with the same inference and post-processing pipeline. No measurable mAP⁵⁰ differences were observed across opset variants, suggesting that opset selection alone is unlikely to explain the accuracy drop in this setup.

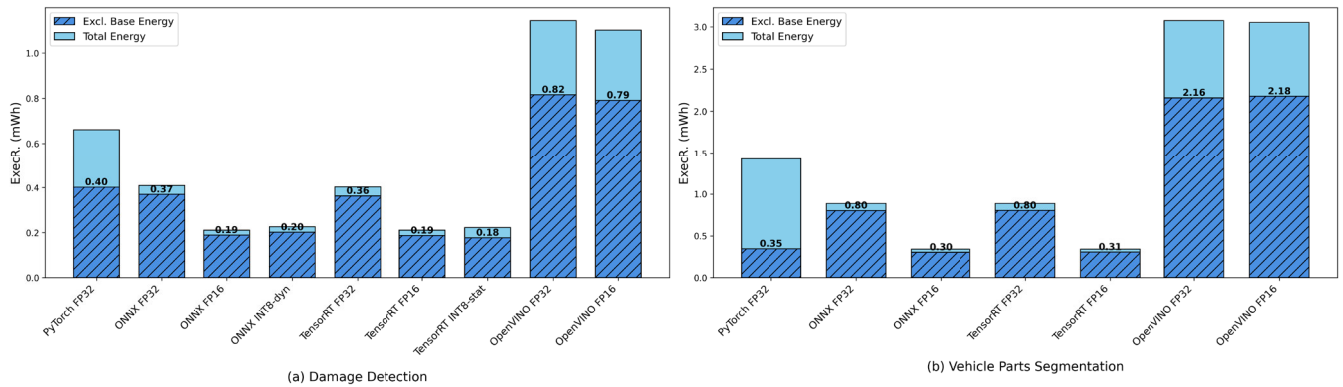


FIGURE 8. Comparative analysis of total energy consumption per request (ExecR, mWh) and baseline-excluded energy consumption per request (mWh) for multiple frameworks on (a) the damage detection task (YOLOv12-S) and (b) the vehicle parts segmentation task (YOLOv11-M).

Future work will focus on isolating the remaining causes of the deployment gap, and on assessing alternative deployment and optimization strategies aimed at reducing mAP degradation without compromising efficiency.

Key Findings: The applied optimization and quantization workflow led to a consistent decrease in accuracy compared with the baseline model. Nevertheless, TensorRT FP16 achieved the best trade-off between accuracy and model size for detection. For segmentation, ONNX FP32 yielded the highest mAP, although at the cost of a substantially larger model size.

2) ENERGY CONSUMPTION

The results in Fig. 8(a) show that both the choice of framework and numerical precision can significantly influence the energy per inference. Among all configurations, OpenVINO in FP32 shows the highest execution energy per request (0.82 mWh) with a total of 1.10 mWh, more than double that of the baseline PyTorch model, which consumes approximately 0.40 mWh. In contrast, optimized inference backends such as TensorRT and ONNX in FP16, reduce the execution energy to nearly half the baseline (0.19 mWh). Furthermore, quantized INT8 models (ONNX dynamic and TensorRT static) also achieve substantial efficiency gains, with energy consumption around 0.20 mWh per request, corresponding to an approximate 65% reduction compared to the PyTorch FP32 baseline.

A similar trend was observed for segmentation of vehicle parts (Fig. 8(b)). OpenVINO in FP16 records the highest energy consumption (2.18 mWh). In contrast, both ONNX and TensorRT in FP16 substantially reduced the energy consumption by more than 75% (≈ 0.30 mWh), with a total execution energy as low as 0.50 mWh. These values are close to the baseline PyTorch model, which requires only 0.35 mWh per request, but results in a considerably higher total energy of approximately 1.5 mWh.

Key Findings: The results indicate that both the framework choice and numerical precision strongly affect energy efficiency. OpenVINO generally consumes the most energy,

whereas optimized backends such as TensorRT and ONNX significantly reduce energy per request. In general, selecting an appropriate framework and precision allows substantial reductions in energy consumption while preserving the model performance in both damage detection and vehicle parts segmentation tasks.

3) INFERENCE TIME

The results in Fig. 9 reveal notable differences in inference efficiency between the frameworks and numerical precisions.

For damage detection, CPU inference times remained relatively high, with ONNX INT8 dynamic exceeding 200 ms, highlighting the limited benefit of dynamic quantization without optimized runtime support. By contrast, OpenVINO FP16 achieved the lowest CPU time (≈ 51 ms), demonstrating its effectiveness in CPU-only environments. In general, the inference time improved compared to the baseline model (152.6 ms). On the GPU, the performance increases dramatically: TensorRT FP16 and INT8 static deliver the fastest results, with inference times below 6 ms, followed closely by ONNX FP32/FP16 (≈ 10 ms). The baseline PyTorch FP32 remained slower (≈ 20 ms) compared to the optimized models.

For segmentation of vehicle parts, CPU execution is even more computationally demanding. The ONNX INT8 dynamic reached 427 ms per request, whereas OpenVINO FP16 provided the best CPU performance (≈ 140 ms). On the GPU, the segmentation inference times decreased significantly, with TensorRT FP16 achieving the best performance (≈ 6 ms), followed by TensorRT FP32 (≈ 13 ms). ONNX FP32/FP16 offers competitive performance (≈ 19 ms), whereas baseline PyTorch FP32 (≈ 33 ms) remains the slowest among the evaluated configurations.

Key Findings: In both tasks, GPU inference consistently outperformed the CPU by a factor of $10\times$ to $50\times$, depending on the framework and precision. TensorRT FP16 and INT8 emerged as the most efficient configurations, enabling real-time inference for both detection and segmentation while maintaining the best trade-off between accuracy and model

size as reported in Section V-E1. For CPU-only scenarios, OpenVINO FP16 represents the most practical solution and, provides the lowest inference time.

F. REAL-WORLD VALIDATION ON PHYSICAL SAMPLES

To validate the practical applicability of the models trained on synthetically generated damage data from the SYNDCAR dataset, a supplementary experiment was conducted. This involved the utilization of four images of the Citroën AMI, captured under natural lighting conditions. Unlike the synthetic training images, these test samples contained physically simulated damage created by attaching adhesive materials and transparent overlays to the vehicle's body and windows. This approach safely reproduced realistic scratches, cracks, broken glass, and broken lights while introducing illumination variability, shadows, and reflections, which are typically absent from synthetic imagery.

In this phase, only the damage detection task was subjected to evaluation, given that the segmentation performance had been previously examined using authentic imagery in antecedent experiments. The inference process was conducted using the optimized YOLOv12-S model (TensorRT FP16), which represents the most efficient configuration identified in the previous benchmarking.

As shown in Fig. 10, the optimized model successfully localized all four types of simulated damage across real physical samples, with detection confidences values ranging from 0.39 to 0.85. Despite the moderate reduction in confidence compared to synthetic conditions, all target classes were recognized correctly. The broken lights class achieved the highest confidence (0.85), while scratches and cracks were consistently detected with accurate bounding boxes. The broken glass class, however, remained the most challenging. Although one instance was correctly identified (0.50), a false negative occurred in the left-side perspective, where the model failed to detect a glass fracture due to specular reflection and low texture contrast between the damaged and undamaged regions.

Overall, these results confirm that a detector trained exclusively on synthetic damage data (SYNDCAR) exhibits promising generalization capabilities under real-world conditions. This experiment represents an encouraging first step toward robust domain generalization, though the observed false negatives highlight the need for larger-scale testing and hybrid fine-tuning with both real and synthetic data to ensure consistent detection across varying viewpoints, lighting, and material conditions.

VI. DISCUSSION AND FUTURE WORK

This study presented SYNDCAR, one of the most significant contributions of this study: a carefully constructed dataset for the detection and localization of exterior vehicle damage. Although not large in scale, SYNDCAR was developed through diverse experiments to explore the potential of synthetic data in such a challenging domain. This direction was chosen after an initial study recently accepted for

publication [7], using a publicly available dataset that revealed limited performance and inconsistent annotations, highlighting the need to build a dedicated and controlled dataset tailored to this application. The dataset provides annotations for both damage detection and vehicle parts segmentation, however, the results indicate that the detection performance still requires substantial improvement. This is largely due to the limited dataset size, the high variability of real-world damage, and the inherent limitation that synthetic images cannot fully replicate the complexity of real conditions. The exclusive focus on a single vehicle model was intended to ensure a controlled and consistent experimental setup. This decision enabled focused validation of the proposed methodology in a realistic use case, allowing the study to demonstrate its technical feasibility and practical applicability before extending it to broader and more diverse scenarios. Despite these challenges, SYNDCAR constitutes a valuable first step toward an ethically viable and controlled benchmark to advance automated vehicle damage assessment.

Extensive experiments were conducted on SYNDCAR, including hyperparameter optimization, transfer learning, benchmarking of CNN- and Transformer-based architectures, and post-training quantization across multiple deployment frameworks and precisions. Importantly, each stage was built upon the insights obtained in the previous stage, allowing for progressive refinement of both the models and the experimental design. The results and lessons learned at each step directly inform the configuration and objectives of the subsequent experiments, strengthening the overall coherence of the study.

In the hyperparameter optimization stage, tuning the critical parameters led to measurable improvements in the overall precision and recall, although not all damage classes benefited equally. This suggests that the impact of hyperparameter adjustment varies depending on the visual characteristics and the imbalance of each class.

Transfer learning and fine-tuning experiments revealed distinct behaviors between architectures: CNN-based models such as YOLOv8 adapted rapidly with minimal tuning, whereas Transformer-based models such as RT-DETR benefited significantly from progressive layer unfreezing, achieving the highest recall and mAP under controlled adaptation.

In the benchmarking phase, the results highlight the trade-offs between the accuracy, inference time, and energy consumption. Larger architectures (e.g., YOLOv8-X) achieved the highest accuracy, whereas YOLOv12-S and YOLOv11-M offered an optimal balance for real-time deployment with reduced energy demand. To ensure a controlled and repeatable benchmark under a single training/inference stack, the evaluation was purposefully limited to the YOLO family and RT-DETR. This prevents additional non-YOLO models from requiring model-specific preprocessing/post-processing and frequently

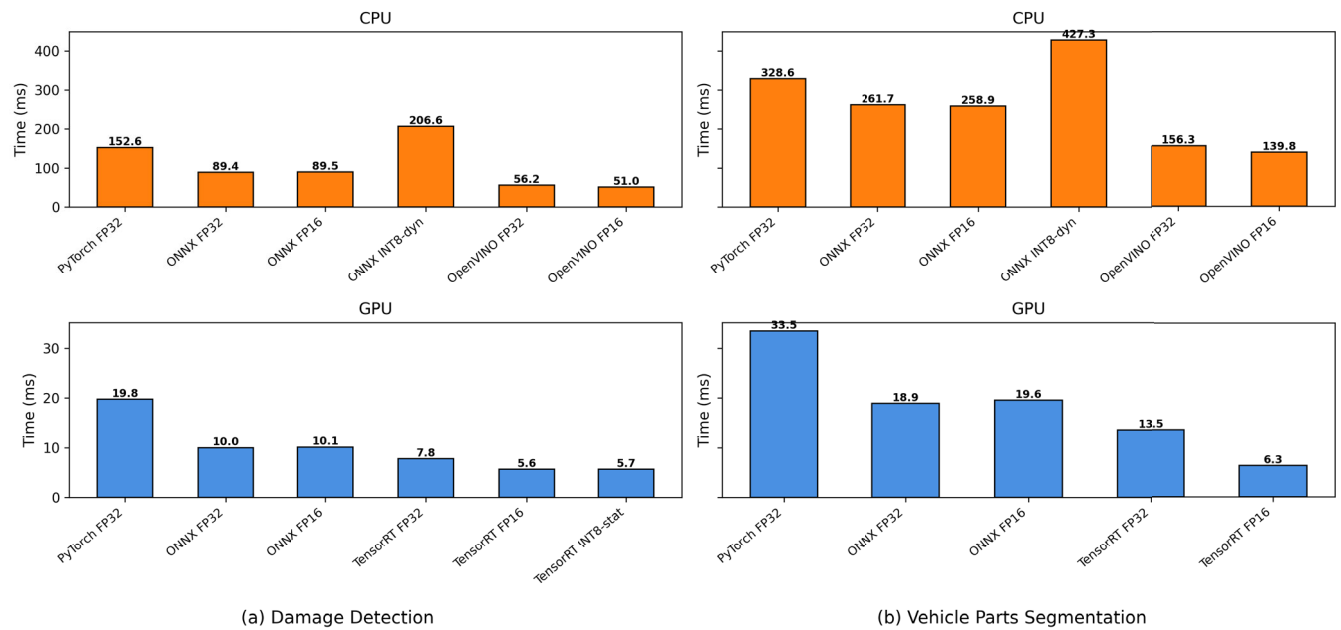


FIGURE 9. Inference time (ms) comparison across deployment frameworks and quantization levels.



FIGURE 10. Qualitative results of the optimized YOLOv12-S (TensorRT FP16) model on four real images of the Citroën AMI with physically simulated damage. Each sub-image corresponds to one perspective.

hardware-dependent optimizations, which can complicate a fair comparison of accuracy–efficiency trade-offs.

Finally, post-training quantization experiments demonstrated that the TensorRT FP16 and INT8 configurations provided up to 65% energy savings and substantial inference speedups, with only moderate accuracy degradation. Both ONNX and TensorRT consistently outperformed the baseline PyTorch implementations, reinforcing their suitability for sustainable smart mobility applications.

Beyond these numerical gains, the results highlight a crucial insight: optimization can effectively reshape the traditional accuracy–complexity trade-off. Larger models are inherently more accurate, but typically incur higher inference times and power consumption. However, through targeted optimization and quantization, it becomes possible to significantly reduce the computational footprint of these complex architectures, while maintaining a competitive

level of accuracy. In practical terms, this implies that an optimized medium-scale model can outperform smaller variants not only in detection accuracy but also in runtime efficiency. For example, in the damage detection task, the YOLOv12-S model deployed with TensorRT FP32 and FP16 achieved inference times of 7.8 and 5.6 ms, respectively, faster than the baseline YOLOv12-N (15 ms). Although quantization reduced the mAP⁵⁰ from 0.830 to 0.518, this value still exceeded that of YOLOv12-N. This shows that even after quantization, optimized higher-capacity models can retain a performance advantage over lighter architectures by combining reduced latency and energy consumption with superior predictive capability.

A brief real-world validation using four images with physically simulated damage confirmed that the optimized YOLOv12-S (TensorRT FP16) model could be generalized from synthetic to real conditions. Despite one missed

detection on a broken glass region, the results are promising and mark the first step toward broader domain generalization.

Looking ahead, several research directions remain open. First, extending SYNDCAR to cover a broader range of vehicle types, colors, and real-world conditions (e.g., different weather, illumination, and occlusion scenarios) will enhance generalization and robustness. Second, benchmarking against a wider variety of transformer-based architectures can provide further insight into their adaptability and comparative advantages in this domain. Third, deploying and rigorously evaluating the optimized models on embedded and edge devices is essential to validate their performance under resource-constrained real-world conditions. Finally, exploring semi-supervised, domain adaptation, and continual learning strategies can help mitigate dataset limitations and support long-term adaptive performance in dynamic urban environments.

VII. CONCLUSION

This study introduced SYNDCAR, a synthetic dataset designed to advance research in automated vehicle damage detection and vehicle parts segmentation. Through comprehensive experimentation, including hyperparameter optimization, transfer learning, model benchmarking, and post-training quantization, the study demonstrated that generative synthetic data can effectively support model training and benchmarking while reducing the ethical and logistical barriers of real-world data collection.

The results reaffirm the inherent trade-offs between accuracy, computational speed, and energy efficiency that characterize modern deep learning models. More broadly, this study demonstrates how synthetic data generation, transfer learning, and model optimization can jointly accelerate the development of computer vision systems that are not only accurate but also deployable under real-world resource constraints. Moving beyond controlled experimentation with synthetic data and providing quantitative insights into the practical feasibility of deploying optimized models in real-world scenarios. This study contributes to the broader goal of enabling scalable, efficient, and sustainable AI solutions for next-generation intelligent mobility and smart-city applications.

REFERENCES

- [1] L. Luo, J. Portugal-Pereira, K. Takami, and G. Parady, "Unintended environmental impacts of private automated vehicles: Insights from gunma prefecture, Japan," *Transp. Res. D, Transp. Environ.*, vol. 134, Sep. 2024, Art. no. 104298, doi: [10.1016/j.trd.2024.104298](https://doi.org/10.1016/j.trd.2024.104298).
- [2] S. Lyu, Y. Huang, and T. Sun, "Urban sprawl, public transportation efficiency and carbon emissions," *J. Cleaner Prod.*, vol. 489, Jan. 2025, Art. no. 144652, doi: [10.1016/j.jclepro.2025.144652](https://doi.org/10.1016/j.jclepro.2025.144652).
- [3] J. Hu, E. Ayaragarnchanakul, Z. Yang, and F. Creutzig, "Shared pooled mobility essential complement to decarbonize China's transport sector until 2060," *Mitigation Adaptation Strategies Global Change*, vol. 29, no. 5, p. 37, Jun. 2024, doi: [10.1007/s11027-024-10135-3](https://doi.org/10.1007/s11027-024-10135-3).
- [4] J. Morfeldt and D. J. A. Johansson, "Impacts of shared mobility on vehicle lifetimes and on the carbon footprint of electric vehicles," *Nature Commun.*, vol. 13, no. 1, p. 6400, Oct. 2022, doi: [10.1038/s41467-022-33666-2](https://doi.org/10.1038/s41467-022-33666-2).
- [5] I. Tikoudis, L. Martinez, K. Farrow, C. G. Bouyssou, O. Petrik, and W. Oueslati, "Exploring the impact of shared mobility services on CO₂," *OECD Environ. Work. Papers*, no. 175, pp. 1–51, 2021, doi: [10.1787/9d20da6c-en](https://doi.org/10.1787/9d20da6c-en).
- [6] Y. Ma, H. Ghanbari, T. Huang, J. Irvin, O. Brady, S. Zalouk, H. Sheng, A. Ng, R. Rajagopal, and M. Narsude, "A system for automated vehicle damage localization and severity estimation using deep learning," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 6, pp. 5627–5639, Jun. 2024, doi: [10.1109/TITS.2023.3334616](https://doi.org/10.1109/TITS.2023.3334616).
- [7] C. Loureiro, S. Dixe, H. Baptista, C. Neto, T. Aston, and D. Fernandes, "AI-driven exterior damage assessment for shared vehicle systems: A benchmarking study of segmentation and detection models," in *Proc. 7th Experiment@ Int. Conf.*, 2025.
- [8] M. J. Hasan, C. K. Nguyen, Y. L. Boo, H. Jahani, and K.-L. Ong, "Vehicle damage detection using artificial intelligence: A systematic literature review," *Wiley Interdiscip. Rev., Data Mining Knowl. Discovery*, vol. 15, no. 2, Jun. 2025, Art. no. e70027, doi: [10.1002/widm.70027](https://doi.org/10.1002/widm.70027).
- [9] N. Dhiab, H. Ghazzai, H. Besbes, and Y. Massoud, "A very deep transfer learning model for vehicle damage detection and localization," in *Proc. 31st Int. Conf. Microelectron. (ICM)*, Dec. 2019, pp. 158–161, doi: [10.1109/ICM48031.2019.9021687](https://doi.org/10.1109/ICM48031.2019.9021687).
- [10] M. Jameel, M. U. I. Arif, A. Hintsches, and L. Schmidt-Thieme, "Automation of leasing vehicle return assessment using deep learning models," in *Machine Learning and Knowledge Discovery in Databases: Applied Data Science Track (ECML PKDD 2020)*. Cham, Switzerland: Springer, 2020, pp. 259–274, doi: [10.1007/978-3-030-67664-3_17](https://doi.org/10.1007/978-3-030-67664-3_17).
- [11] K. Thonglek, N. Urailetpasert, P. Pattiyathane, and C. Chantrapornchai, "Vehicle part damage analysis platform for autoinsurance application," *ECTI Trans. Comput. Inf. Technol. (ECTI-CIT)*, vol. 15, no. 3, pp. 313–323, Sep. 2021, doi: [10.37936/ecti-cit.2021153.223151](https://doi.org/10.37936/ecti-cit.2021153.223151).
- [12] M. Venkatesh, R. Oruganti, S. A. Katti, K. Subramanian, and T. S. Chandar, "Vehicle damage detection using machine learning and segmentation to aid insurance claims," in *Proc. IEEE 20th India Council Int. Conf. (INDICON)*, Dec. 2023, pp. 932–938, doi: [10.1109/INDICON59947.2023.10440856](https://doi.org/10.1109/INDICON59947.2023.10440856).
- [13] O. Amodu, A. Shaban, and G. Akinade, "Revolutionizing vehicle damage inspection: A deep learning approach for automated detection and classification," in *Proc. 9th Int. Conf. Internet Things, Big Data Secur.*, Sep. 2024, pp. 199–208, doi: [10.5220/0012630700003705](https://doi.org/10.5220/0012630700003705).
- [14] X. Wang, W. Li, and Z. Wu, "CarDD: A new dataset for vision-based car damage detection," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 7, pp. 7202–7214, Jul. 2023, doi: [10.1109/TITS.2023.3258480](https://doi.org/10.1109/TITS.2023.3258480).
- [15] C. Loureiro, S. Dixe, C. Neto, A. Silva, B. Miranda, R. Rodrigues, N. Costa, D. Ferreira, C. Ferreira, J. Machado, and D. Fernandes, "SYNDCAR: Synthetic vehicle damage detection and parts segmentation dataset," Mendeley Data, Guimarães, Portugal, Tech. Rep. V1, 2025, doi: [10.17632/hzpj48krdt.1](https://doi.org/10.17632/hzpj48krdt.1).
- [16] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-resolution image synthesis with latent diffusion models," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, New Orleans, LA, USA, Jun. 2022, pp. 10674–10685.
- [17] N. Sattler. *OneTrainer*. GitHub repository. Accessed: Aug. 1, 2025. [Online]. Available: <https://github.com/Nerogar/OneTrainer>
- [18] GitHub Repository. *Stable Diffusion Web UI*. Accessed: Aug. 1, 2025. [Online]. Available: <https://github.com/AUTOMATIC1111/stable-diffusion-webui>
- [19] GitHub Repository. *Computer Vision Annotation Tool (CVAT)*. Accessed: Aug. 1, 2025. [Online]. Available: <https://github.com/cvat-ai/cvat>
- [20] B. Moore, J. Philbin, and J. Corso. *FiftyOne: The Open-Source Tool for Building High-Quality Datasets and Computer Vision Models*. Accessed: Aug. 1, 2025. [Online]. Available: <https://github.com/voxel51/fiftyone>
- [21] M. Nevendra and P. Singh, "Empirical investigation of hyperparameter optimization for software defect count prediction," *Expert Syst. Appl.*, vol. 191, Apr. 2022, Art. no. 116217, doi: [10.1016/j.eswa.2021.116217](https://doi.org/10.1016/j.eswa.2021.116217).
- [22] *Ultralytics: YOLO and Other Cutting-Edge Models*. Accessed: Oct. 3, 2025. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [23] *Ray Tune: Scalable Hyperparameter Tuning*. Accessed: Sep. 27, 2025. [Online]. Available: <https://docs.ray.io/en/latest/tune/index.html>
- [24] I. B. Akkaya, S. S. Kathiresan, E. Arani, and B. Zonooz, "Enhancing performance of vision transformers on small datasets through local inductive bias incorporation," *Pattern Recognit.*, vol. 153, Sep. 2024, Art. no. 110510, doi: [10.1016/j.patcog.2024.110510](https://doi.org/10.1016/j.patcog.2024.110510).

- [25] M. Kotthapalli, D. Ravipati, and R. Bhatia, "YOLOv1 to YOLOv11: A comprehensive survey of real-time object detection innovations and challenges," 2025, *arXiv:2508.02067*.
- [26] A. Wang, H. Chen, L. Liu, K. Chen, Z. Lin, J. Han, and G. Ding, "YOLOv10: Real-time end-to-end object detection," 2024, *arXiv:2405.14458*.
- [27] R. Khanam and M. Hussain, "YOLOv11: An overview of the key architectural enhancements," 2024, *arXiv:2410.17725*.
- [28] *YOLO11: Next-Generation Real-Time Object Detection Models*. Accessed: Oct. 23, 2025. [Online]. Available: <https://docs.ultralytics.com/pt/models/yolo11/>
- [29] Y. Tian, Q. Ye, and D. Doermann, "YOLOv12: Attention-centric real-time object detectors," 2025, *arXiv:2502.12524*.
- [30] *YOLO12: Attention-Centric Real-Time Object Detection Models*. Accessed: Oct. 23, 2025. [Online]. Available: <https://docs.ultralytics.com/pt/models/yolo12/>
- [31] Y. Zhao, W. Lv, S. Xu, J. Wei, G. Wang, Q. Dang, Y. Liu, and J. Chen, "DETRs beat YOLOs on real-time object detection," 2023, *arXiv:2304.08069*.
- [32] *RT-DETR: Real-Time DEtection TRansformer Models*. Accessed: Oct. 23, 2025. [Online]. Available: <https://docs.ultralytics.com/pt/models/rtdetr/>
- [33] J. Yang, X. Shen, J. Xing, X. Tian, H. Li, B. Deng, J. Huang, and X. Hua, "Quantization networks," 2019, *arXiv:1911.09464*.
- [34] Sumaiya, R. Jafarpourmarzouni, Y. Luo, S. Lu, and Z. Dong, "Toward real-time and efficient perception workflows in software-defined vehicles," *IEEE Internet Things J.*, vol. 12, no. 6, pp. 7240–7258, Mar. 15, 2025, doi: [10.1109/JIOT.2024.3492801](https://doi.org/10.1109/JIOT.2024.3492801).
- [35] D. K. Alqahtani, A. Cheema, and A. N. Toosi, "Benchmarking deep learning models for object detection on edge computing devices," 2024, *arXiv:2409.16808*.
- [36] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16×16 words: Transformers for image recognition at scale," 2020, *arXiv:2010.11929*.



Transformation CoLab, where she develops advanced AI and computer vision solutions to accelerate the digital transformation and automation of industrial processes.

CÁTIA LOUREIRO received the master's degree in biomedical engineering, with a specialization in medical instrumentation and informatics from the University of Trás-os-Montes and Alto Douro (UTAD), in 2023. She began her research career in the field of computer vision with her master's thesis and later continued as a Research Assistant in a project focused on developing AI solutions for the veterinary field. She is currently a Computer Vision Researcher with DTx–Digital



the automotive industry, as well as on industry 4.0 technologies, with an emphasis on digital twins and software-defined radio. She has published work in these areas.

SANDRA DIXE received the master's degree in industrial electronics and computer engineering from the University of Minho, with a specialization in automation, control, robotics, and power systems. She is currently a Computer Vision Researcher with DTx–Digital Transformation CoLAB. She is also an Invited Assistant Professor with the Polytechnic Institute of Cávado and Ave (IPCA). Her research interests include computer vision, particularly deep learning for



index of 10) in the areas of machine learning, knowledge engineering, and interoperability.

CRISTIANA NETO received the Ph.D. degree in biomedical engineering (medical informatics) from the University of Minho, Portugal, in 2024. Since 2019, she has been an Adjunct Assistant Professor with the University of Minho. She has been a member of the Knowledge Engineering Group (KEG), Algoritmi Research Centre, since 2018. She has participated in several innovative research projects as a Research Fellow. Her scientific output includes over 30 publications (H-



initially with Neadvance (worked on projects, such as Defect Detection on Specular Surfaces and the PT2020 co-promotion project with PSA Mangualde, developing a computer vision solution for paint quality control), and later with DTx. His work focuses on the research and development of defect inspection algorithms and optical quality analysis systems for industrial applications, as well as the implementation of machine learning and artificial intelligence algorithms. He has contributed to several projects, notably for Bosch (Laser Finishing Quality Inspection and Spear) and IKEA (Digital Quality: Design Quality Concept Applied to Filler). In 2025, he completed a specialization course in deep learning with the Instituto Superior Técnico (Técnico+).

ANTÓNIO SILVA received the bachelor's degree in electronic and computer engineering from the Universidade Lusíada of Vila Nova de Famalicão and the integrated master's degree in electrical and computer engineering from the Faculty of Engineering of the University of Porto (FEUP). He has attended the Biomedical Image Analysis module with FEUP. He is a Computer Vision Researcher with the Computer Graphics and Vision Group, DTx. With eight years of experience, he was



has been involved in several research and development projects, including the development of an augmented reality solution for training new industrial operators, contributing to the adoption of immersive technologies in digital transformation processes.

BEATRIZ MIRANDA received the integrated master's degree in biomedical engineering, with a specialization in medical electronics from the University of Minho, in 2022. She began her research career in the field of extended reality applied to industrial and healthcare contexts, with her master's thesis focusing on the use of augmented reality for motor training and assessment of Parkinson's disease patients. Since joining DTx–Digital Transformation CoLAB in 2023, she



RICARDO RODRIGUES (Member, IEEE) received the Ph.D. degree in computer science from the University of Porto. He is currently an Auxiliary Researcher with DTx and an Invited Professor, focusing on cyber-physical systems, distributed AI, and intelligent control for smart manufacturing. He has contributed to several EU research and development projects. He has over 30 publications. He is a member of IFAC committees. He received the Industry 4.0 Award.



NUNO M. C. DA COSTA received the integrated master's degree in biomedical engineering from the Faculty of Science and Technology, New University of Lisbon, in 2012, and the joint Ph.D. degree in bioengineering (neurosciences and data science) from the ICVS, Algoritmi, 2AI, University of Minho, and MIT, in 2021. He was an Invited Coordinating Researcher with the Motionlab, Algoritmi Center, University of Minho, Azurém Campus. He is currently a Senior

Data Scientist and a Coordinator of the Data Science and Machine Learning Group, Collaborative Laboratory Association in Digital Transformation (DTx CoLab). He is also a Visiting Professor with IPCA, Barcelos. He has published six articles in journals as the first author, is the author of one book, and has published 15 articles as a co-author. In his curriculum, the most frequent terms in the context of scientific and technological production include data science, machine learning, GENAI, programming, Python, high-performance computing, neuroscience, HCI, BCI, biostatistics, mathematics, physics, electronics, instrumentation, biodevices, and biomedical. He has received one award/distinction.



CARLOS FERREIRA (Member, IEEE) received the master's degree in electronics engineering from the University of Minho, Portugal, in 2016, where he is currently pursuing the Ph.D. degree in advanced engineering systems for industry. His research is centered on inertial and optical MEMS design and characterization combined with electronic design for closed-loop control and high-speed test, using FPGA and ARM-based embedded systems.

HUGO PEIXOTO received the Ph.D. degree in biomedical engineering. He is currently an Assistant Researcher with the Algoritmi Research Centre, University of Minho. Active in research since 2009, he has authored over 90 indexed publications (H-index of 11), with a scope that ranges from interoperability to AI and knowledge engineering, and has participated in international projects. He serves as a Guest Editor for a Special Issue on *Machine Learning in Healthcare*.

JOSÉ MACHADO is currently a Full Professor with the University of Minho. Previously, he has been with the University of Minho, since 1988, focusing on research and development applied to healthcare, justice, and public services. He co-authored the award-winning AIDA platform used in Portuguese hospitals and contributed to European Diploma Supplement. He held leadership roles with the Algoritmi and CCTC, coordinated ERASMUS in medical informatics, and supervised over 120 students. With more than 470 publications, 2220 Scopus citations, and an H-index of 24, he is active in international research, editorial boards, and conference organization, and has evaluated projects for institutions across several countries.



DUARTE FERNANDES received the master's and Ph.D. degrees in electronic and computer engineering, specializing in industrial informatics and embedded systems from the University of Minho. He is currently the Coordinator with the Computer Graphics and Vision Group, DTx CoLab, which develops its research and development activities in the field of immersive technologies and artificial vision for a consortium of 13 companies.

• • •